Architecture et ressources déployées

Environnements, reseau, execution et livraison continue

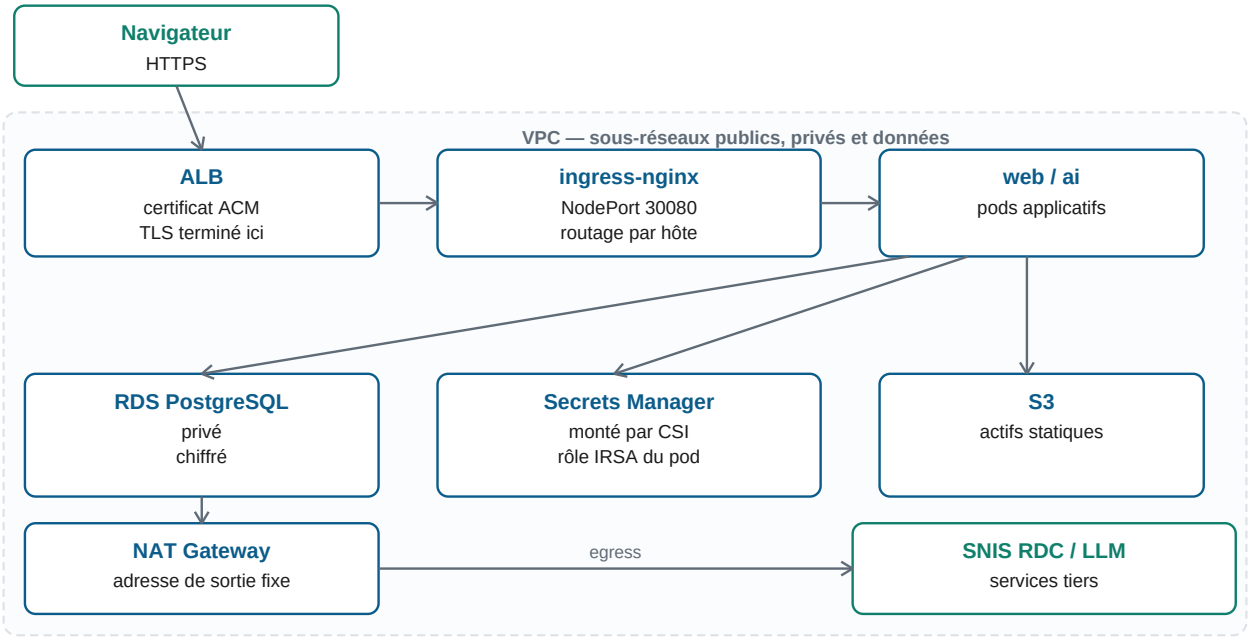
Plateforme SIVAX — infrastructure Amazon Web Services
Document établi à partir de l'inventaire du compte AWS.

Version du 9 septembre 2026. Ce document décrit la plateforme SIVAX telle qu'elle est déployée sur Amazon Web Services, et les ressources qui la composent. Il a été établi à partir de l'inventaire du compte AWS, non d'un schéma d'intention : chaque élément cité a été relevé sur les environnements en service.

Ce que la plateforme dessert

SIVAX met à disposition des décideurs du secteur de la santé en République démocratique du Congo une lecture consolidée des données du SNIS RDC, un assistant conversationnel et un système d'alertes. L'infrastructure décrite ici porte trois charges de travail : l'application web, le service d'intelligence artificielle, et les travaux planifiés qui synchronisent les données.

Vue d'ensemble



Chemin d'une requête, de gauche à droite, et dépendances vers le bas.

Une requête entre par un load balancer applicatif qui termine le chiffrement TLS avec un certificat géré par AWS. Elle est transmise au contrôleur d'ingress qui tourne dans le cluster Kubernetes, lequel la route vers l'application selon le nom de domaine appelé. L'application interroge la base de données, lit ses secrets et sert ses actifs statiques ; ses appels vers l'extérieur — SNIS RDC, fournisseur de modèles de langage — sortent par une passerelle NAT dont l'adresse est fixe.

Les deux environnements

Le même code d'infrastructure décrit les deux environnements ; seuls le dimensionnement et les interrupteurs de sûreté diffèrent. Chacun conserve son propre état, de sorte qu'une erreur commise sur l'un ne peut pas atteindre l'autre.

	Développement	Production
Région	eu-west-3 (Paris)	eu-central-1 (Francfort)
Nom public	dev.sivax.health	www.sivax.health
Réseau privé	10.20.0.0/16	10.40.0.0/16
Zones de disponibilité	2	3

	Développement	Production
Cluster Kubernetes	1.31	1.31
Nœuds	2 × t3a.medium	2 × t3a.medium + 2 × t3a.large
Base de données	PostgreSQL 16.13, db.t4g.micro	PostgreSQL 16.13, db.t4g.small
Sauvegardes	7 jours	14 jours
Branche déployée	dev	prod

Le choix de deux régions distinctes n'est pas cosmétique : il garantit qu'un incident régional touchant l'une laisse l'autre intacte, et il évite que les deux environnements se disputent les mêmes quotas de service.

Le réseau et son cloisonnement

Chaque environnement dispose de son propre réseau privé, découpé en trois catégories de sous-réseaux dont les droits diffèrent :

Catégorie	Contenu	Accès depuis Internet
Publics	load balancer, passerelles NAT	entrant sur 80 et 443 uniquement
Privés	nœuds du cluster et pods	aucun ; sortie par la NAT
Données	base de données	aucun, dans les deux sens

Le cloisonnement repose sur l'absence de route, non sur une règle de filtrage : un service placé dans les sous-réseaux de données n'a aucun chemin vers Internet, ce qui le rend structurellement injoignable plutôt que conditionnellement protégé.

Toute sortie vers l'extérieur emprunte une passerelle NAT dont l'adresse publique est fixe. C'est cette propriété qui permet de demander au ministère l'inscription de la plateforme sur une liste d'autorisation SNIS RDC : sans elle, l'adresse changerait au gré du remplacement des machines.

Le bord public

Le point d'entrée est un load balancer applicatif, un par environnement, avec un certificat émis et renouvelé automatiquement par AWS Certificate Manager. Le chiffrement s'arrête là ; à l'intérieur du réseau privé, le trafic circule en clair entre le load balancer et le contrôleur d'ingress.

Le certificat de production couvre à la fois `sivax.health` et `www.sivax.health`. Les noms publics restent gérés chez l'hébergeur du domaine, qui porte également la messagerie.

Exécution des applications

Les charges de travail tournent sur Amazon EKS, le service Kubernetes managé d'AWS. Chaque cluster porte les mêmes composants complémentaires, installés en tant qu'extensions gérées par AWS :

Composant	Rôle
vpc-cni	attribue à chaque pod une adresse du réseau privé
coredns	résolution de noms interne au cluster

Composant	Rôle
kube-proxy	routage des services internes
metrics-server	mesures de charge, base de l'ajustement automatique
aws-ebs-csi-driver	volumes persistants

Les charges de travail déployées sont les suivantes :

Charge	Nature	Déclenchement
web	application Next.js	permanent
ai	service Python d'assistance	permanent
redis	cache en cluster	permanent
etl-incremental	synchronisation SNIS RDC	chaque nuit à 03:00 UTC
etl-full	synchronisation complète	chaque dimanche à 04:00 UTC
db-migrate	migration de schéma	à la demande

En production, les modules du cluster et les applications sont séparés sur deux groupes de nœuds distincts, de sorte qu'une application qui consommerait excessivement ne puisse pas gêner les fonctions vitales du cluster.

Identité et secrets

Aucune charge de travail ne détient de clé d'accès AWS. Chaque application assume un rôle IAM qui lui est propre, associé à son compte de service Kubernetes : le cluster présente un jeton signé, AWS vérifie son origine et délivre des identifiants temporaires. Il n'existe donc aucun identifiant permanent à faire tourner ni à révoquer.

Rôle	Charge servie	Droits
role-diav1-web-*	application web	lecture des secrets, écriture des actifs
role-diav1-ai-*	service d'assistance	lecture des secrets et des actifs
role-diav1-jobs-*	travaux planifiés	lecture des secrets, écriture des actifs
role-diav1-cicd-*	chaîne de livraison	poussée d'images dans le registre

Les secrets — chaîne de connexion à la base, clés d'API tierces, clés de signature — sont conservés dans AWS Secrets Manager, dix par environnement. Ils sont montés dans les pods au démarrage par un pilote dédié, sous le rôle du pod qui les demande, et n'apparaissent dans aucun fichier du dépôt.

Deux d'entre eux ne sont connus de personne : le mot de passe administrateur de la base de données et la clé de chiffrement des paramètres applicatifs sont générés à la création de l'environnement et ne sont jamais affichés.

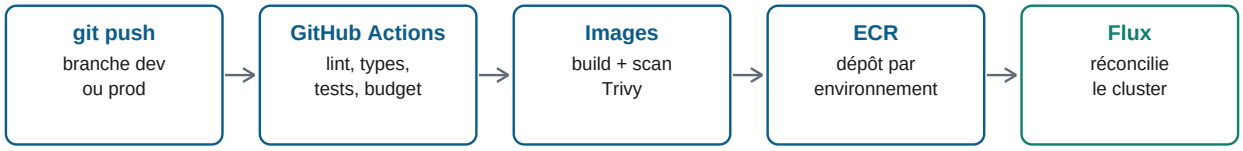
Données

Base de données. PostgreSQL 16.13, en instance gérée, sans adresse publique et chiffrée au repos. La capacité augmente d'elle-même en cas de besoin — de 20 à 100 Go en développement, de 50 à 200 Go en production — ce qui évite l'incident d'un disque plein prévisible. Les sauvegardes sont automatiques et conservées 7 jours en développement, 14 en production.

Actifs statiques. Un bucket S3 par environnement porte les fonds de carte, les images et les exportations. Une règle de cycle de vie déplace les artefacts anciens vers un stockage moins coûteux, puis les supprime.

Cache. Redis tourne dans le cluster plutôt qu'en service géré. Ce qu'il contient — métadonnées SNIS RDC, résultats d'analyse, compteurs de limitation de débit — est intégralement reconstructible, et l'application sait fonctionner sans lui. Le service géré équivalent reste disponible par simple changement de paramètre, si la durabilité venait à le justifier.

Livraison continue



Le déploiement est un commit : le pipeline déplace le tag d'image, Flux applique.
Aucun accès au cluster depuis le pipeline — le dépôt reste la seule source de vérité.

Un développeur pousse son travail sur la branche de l'environnement visé. La chaîne de livraison, hébergée par GitHub Actions, exécute d'abord les contrôles de qualité : vérification des types, analyse statique, budget de poids des actifs statiques, résolution des dépendances et tests du service Python. Elle construit ensuite les deux images, les analyse à la recherche de vulnérabilités connues, et ne les publie au registre que si l'analyse est propre.

Le déploiement proprement dit est un commit : la chaîne déplace la référence d'image dans le dossier de l'environnement, et l'agent Flux installé dans le cluster applique ce changement dans les deux minutes. Le pipeline n'a donc aucun accès au cluster, et le dépôt reste la seule description de ce qui tourne.

La chaîne s'authentifie auprès d'AWS par fédération d'identité : elle présente un jeton signé par GitHub, borné à ce dépôt et à cette branche. **Aucune clé d'accès AWS n'est stockée dans GitHub.**

Durée mesurée d'un déploiement complet, du `git push` à l'application en service : **8 minutes 16 secondes** en production, dont 5 minutes 30 de construction et de contrôles.

Observabilité

Chaque environnement dispose de journaux centralisés dans CloudWatch, avec une rétention de 30 jours, et de quatre alarmes :

Alarme	Ce qu'elle surveille
rds-cpu	processeur de la base au-delà de 80 % pendant 15 minutes
rds-storage	espace disque restant sous le seuil
rds-connections	connexions proches du plafond, signe d'une fuite
etl-no-success	aucune synchronisation réussie depuis 24 heures

La dernière est la plus spécifique à cette plateforme : une synchronisation qui échoue en silence dégrade tous les tableaux de bord sans qu'aucune erreur n'apparaisse ailleurs.

Les applications émettent par ailleurs leurs traces et leurs mesures vers AWS X-Ray et CloudWatch, par un collecteur OpenTelemetry embarqué dans chaque pod.

Inventaire des ressources

Ressource	Développement	Production
Cluster Kubernetes	1	1
Nœuds	2	4
Base de données	1	1
Load balancer applicatif	1	1
Passerelles NAT	1	3
Adresses IPv4 publiques	3	6
Sous-réseaux	6	9
Dépôts d'images	2	2
Buckets S3	2	2
Secrets	10	10
Groupes de journaux	5	5
Alarmes	4	4
Rôles IAM applicatifs	4	4

S'y ajoute, pour l'ensemble du compte, un bucket S3 conservant l'état de l'infrastructure et une table de verrous qui empêche deux modifications simultanées.

Contraintes du compte et leurs conséquences

Le compte AWS est récent et n'a pas encore été vérifié par AWS. Trois services sont de ce fait refusés, ce qui a orienté certains choix de conception :

Service refusé	Message d'AWS	Conséquence retenue
Network Load Balancer	*This AWS account currently does not support creating load balancers*	le bord est un load balancer applicatif, qui apporte en outre le certificat géré
CloudFront	*Your account must be verified before you can add new CloudFront resources*	pas de réseau de diffusion de contenu ; les actifs sont servis directement
Exécution CodeBuild	*Cannot have more than 0 builds in queue for the account*	la chaîne de livraison s'exécute sur GitHub Actions

Une seule demande auprès du support AWS lève les trois. La plateforme n'en dépend pas : elle fonctionne aujourd'hui dans ces limites, et chacune des options écartées reste préparée dans le code, activable par un

paramètre.

Points ouverts

Trois éléments méritent une décision, sans caractère bloquant :

- **Personne ne reçoit les alertes.** Les quatre alarmes de chaque environnement publient vers un canal de notification auquel aucun destinataire n'est abonné, et aucun budget de dépense n'est actif. Renseigner une ou plusieurs adresses de courriel suffit à activer les deux.
- **Le nom racine du domaine** ne pointe pas encore vers la plateforme. La norme DNS interdisant l'alias employé pour le sous-domaine, il est prévu de le traiter par une redirection vers `www.sivax.health`.
- **Le dimensionnement de la production** est confortable au regard de la charge observée. Une réduction est possible ; elle est chiffrée dans le document d'évaluation des coûts.